Суть лаби порівняти ***алгоритми оптимізації*** SGD, Adam, RMSProp для навчання невеликої нейронної мережі

SGD - оновлює ваги моделі, використовуючи градієнти, які розраховуються на основі одного випадкового прикладу (або маленької групи даних – mini-batch).

Adam - зберігає "пам’ять" про попередні градієнти, щоб рухатися швидше в потрібному напрямку та адаптує швидкість навчання для кожного параметра окремо, залежно від того, як змінюється градієнт.

RMSProp - динамічно змінює швидкість навчання для кожного параметра, враховуючи середнє значення градієнтів у квадраті. Це допомагає уникати коливань і швидше знаходити оптимальні значення.

Застосувавши ***методи регуляризації*** L1 L2 Dropout (ну взагалі це такий певний спосіб додати додаткове обмеження до моделі з метою щоб уникнути перенавчання (overfitting).)

L1 регулярізація: Додає до функції втрат суму абсолютних значень всіх ваг (коефіцієнтів). (Застосовується, коли потрібно "вимкнути" непотрібні нейрони (зробити їх ваги рівними нулю).

L2 регулярізація: Додає до функції втрат суму квадратів ваг. (Розподіляє "штраф -" рівномірно по всіх вагах, зменшуючи їх значення, але зазвичай не робить їх рівними нулю.)

Dropout — випадковим чином "відключає" (встановлює в 0) деякі нейрони під час кожного кроку навчання. Dropout допомагає запобігти перенавчанню. Ну і типу це сприяє тому, що модель вчиться бути більш узагальненою.

Scheduler — механізм, який автоматично змінює швидкість навчання (learning rate) протягом процесу тренування за певною схемою, у моєму коді, я використовував

- **StepLR:** Зменшує learning rate на певний коефіцієнт після кожного певного числа епох.
- **CosineAnnealingLR(косайн-анилинь):** Змінює learning rate за косинусною кривою, плавно знижуючи його поступово наближаючись до нуля.
- **ExponentialLR:** Експоненційно зменшує learning rate з кожною епохою.

Нормалізація — це процес приведення даних до спільного стандарту або масштабу. Наприклад, значення можуть бути перетворені так, щоб вони знаходилися у діапазоні від 0 до 1 або мали середнє 0 і стандартне відхилення 1.

Я використовую **Batch Normalization** – тобто це нормалізація, яка допомагає стабілізувати та прискорити навчання нейронної мережі.

Що робить:

Batch Normalization нормалізує (стандартизує) виходи (активації) шару за допомогою обчислення середнього значення і стандартного відхилення для кожного міні-батчу даних. Потім ці значення використовуються для того, щоб "перетворити" активації: відняти середнє і поділити на стандартне відхилення. Таким чином, виходи мають приблизно середнє 0 і стандартне відхилення 1.

Навіщо потрібне:

1. **Стабільність навчання:** Нормалізація зменшує внутрішнє зсув (тобто зміни в розподілі даних на вході до кожного шару), що робить навчання більш стабільним.
2. **Прискорення збіжності:** Мережа швидше сходиться до оптимальних параметрів, оскільки градієнти стають більш стабільними і не “збіжуються” занадто повільно або надто швидко.
3. **Регуляризація:** Batch Normalization може діяти як форма регуляризації, що зменшує перенавчання.

Що до графіків

SGD (Stochastic Gradient Descent)

- Повільніший старт : на перших епохах втрата (loss) більша, а точність (accuracy) нижча, ніж у Adam/RMSprop.
- Без додаткового налаштування (наприклад, сильнішого зменшення learning rate або більшої кількості епох) часто дає трохи нижчу кінцеву точність, що видно з усіх трьох scheduler’ів.
- Після 5–6 епох (коли для StepLR чи ExponentialLR зменшується learning rate або коли CosineAnnealingLR “згладжує” lr) помітне зниження втрат і поступове зростання точності. Однак у межах 10 епох Adam та RMSprop зазвичай встигають навчитися краще.

Adam

- швидше знаходить мінімум, тому вже з перших епох має менші втрати і вищу точність порівняно з SGD.
- У більшості експериментів виходить на ~0.95–0.96 тестової точності до 10-ї епохи.
- Залежно від графіка learning rate (scheduler) може трохи швидше або повільніше поліпшуватися, але зазвичай результати досить стабільні.

RMSprop

- за характеристиками подібний до Adam бо також є адаптивним оптимізатором
- У деяких випадках (наприклад, з CosineAnnealingLR чи ExponentialLR) RMSprop досягає навіть трохи вищої або схожої точності, ніж Adam (0.96+).
- Швидко знижує втрати на початку навчання, особливо в перші 3–5 епох, а потім поступово дорівнює результатам Adam.

1. **Підготовка даних:**
   - Завантаження MNIST.
   - Вибір підмножини (10000 тренувальних та 2000 тестових зразків) для швидкого експерименту.
   - Створення DataLoader-ів для зручного пакетного завантаження даних.

```
# ============================
# 1. Підготовка даних (Dataset)
# ============================

# Створюємо послідовність перетворень для зображень: конвертуємо зображення в тензор
transform = transforms.Compose([transforms.ToTensor()])

# Завантажуємо датасет MNIST
# Параметри:
#   - root: шлях до папки з даними;
#   - train: True для тренувального набору, False для тестового;
#   - download: завантажити, якщо дані відсутні;
#   - transform: застосувати перетворення до зображень.
train_dataset = datasets.MNIST(root='./data', train=True, download=True, transform=transform)
test_dataset  = datasets.MNIST(root='./data', train=False, download=True, transform=transform

# Для пришвидшення експериментів обираємо підмножину даних:
#   - 10000 зразків для тренування
#   - 2000 зразків для тестування
train_indices = np.random.choice(len(train_dataset), 10000, replace=False)
test_indices  = np.random.choice(len(test_dataset), 2000, replace=False)
train_subset = Subset(train_dataset, train_indices)
test_subset  = Subset(test_dataset, test_indices)

# Створюємо DataLoader-и для пакетного завантаження даних.
# batch_size=64 – кількість зразків в одному пакеті.
# shuffle=True для тренувального набору – щоб дані були перемішані.
train_loader = DataLoader(train_subset, batch_size=64, shuffle=True)
test_loader  = DataLoader(test_subset, batch_size=64, shuffle=False)
```

2. **Визначення моделі (NeuralNet):**
    - **fc1, fc2, fc3:** Полносвязні шари для перетворення вхідних даних.
    - **BatchNorm1d (bn1, bn2):** Нормалізують виходи перших двох шарів, стабілізують навчання.
    - **Dropout (dropout1, dropout2):** Випадкове "відключення" нейронів (50%) для зменшення перенавчання.
    - **forward:** Метод, який визначає послідовність операцій, що виконується над даними (перетворення вектора, застосування активацій, нормалізація та Dropout).

```
# Створюємо клас нейронної мережі, наслідуючи nn.Module
class NeuralNet(nn.Module):
    def __init__(self):
        super(NeuralNet, self).__init__()
        # Перший полносвязний шар:
        #   - Вхід: зображення 28x28 перетворюється на вектор розміром 784.
        #   - Вихід: 256 нейронів.
        self.fc1 = nn.Linear(28*28, 256)
        # Batch Normalization для першого шару, нормалізує 256 виходів.
        self.bn1 = nn.BatchNorm1d(256)
        # Dropout для першого шару: випадкове "відключення" 50% нейронів.
        self.dropout1 = nn.Dropout(0.5)

        # Другий полносвязний шар:
        #   - Вхід: 256 нейронів
        #   - Вихід: 128 нейронів
        self.fc2 = nn.Linear(256, 128)
        # Batch Normalization для другого шару, нормалізує 128 виходів.
        self.bn2 = nn.BatchNorm1d(128)
        # Dropout для другого шару: також 50%.
        self.dropout2 = nn.Dropout(0.5)

        # Вихідний шар:
        #   - Вхід: 128 нейронів
        #   - Вихід: 10 нейронів (один для кожного класу від 0 до 9)
        self.fc3 = nn.Linear(128, 10)
```

```python
def forward(self, x):
    # Перетворення зображення (розмір [batch_size, 1, 28, 28])
    # у вектор розміром [batch_size, 784]
    x = x.view(-1, 28*28)
    # Проходження через перший шар
    x = self.fc1(x)
    # Нормалізація виходів першого шару
    x = self.bn1(x)
    # Активація ReLU
    x = torch.relu(x)
    # Застосування Dropout для зменшення перенавчання
    x = self.dropout1(x)

    # Проходження через другий шар
    x = self.fc2(x)
    # Нормалізація виходів другого шару
    x = self.bn2(x)
    # Активація ReLU
    x = torch.relu(x)
    # Застосування Dropout
    x = self.dropout2(x)

    # Проходження через вихідний шар
    x = self.fc3(x)
    return x
```

3. **Функція навчання (train_model):**
    - Виконується прямий прохід, обчислення втрат (CrossEntropyLoss + L1 регулярізація, якщо задана), зворотне розповсюдження помилки і оновлення ваг.
    - Після кожної епохи викликається `scheduler.step()`, що змінює швидкість навчання відповідно до обраної схеми.
    - Проводиться оцінка моделі на тестових даних для обчислення точності та втрат.

```
def train_model(model, train_loader, test_loader, optimizer, scheduler,
epochs=10, l1_lambda=0.0):
    # Визначаємо функцію втрат. Для класифікації використовуємо
CrossEntropyLoss.
    criterion = nn.CrossEntropyLoss()

    # Ініціалізуємо списки для збереження значень втрат і точності
    train_losses = []
    test_losses  = []
    train_acc    = []
    test_acc     = []

    # Проходимо по заданій кількості епох
    for epoch in range(epochs):
        # Переводимо модель у режим тренування (включає Dropout, BatchNorm
в режимі навчання)
        model.train()
        running_loss = 0.0  # Накопичені втрати за епоху
        correct = 0         # Кількість правильних передбачень
        total = 0           # Загальна кількість зразків

        # Обробка даних по батчах
        for data, target in train_loader:
            # Обнуляємо градієнти оптимізатора
            optimizer.zero_grad()
            # Прямий прохід: отримуємо передбачення моделі
            outputs = model(data)
            # Обчислюємо основну втрату
            loss = criterion(outputs, target)

            # Якщо заданий параметр l1_lambda більше нуля, застосовуємо L1
регулярізацію
            if l1_lambda > 0:
                # Обчислюємо суму абсолютних значень ваг
                l1_norm = sum(p.abs().sum() for p in model.parameters())
                # Додаємо штраф до основної втрати
                loss = loss + l1_lambda * l1_norm

            # Зворотне розповсюдження помилки
            loss.backward()
            # Оновлення ваг моделі
            optimizer.step()

            # Накопичення втрат для розрахунку середнього значення
            running_loss += loss.item() * data.size(0)
            # Отримуємо передбачені класи
            _, predicted = torch.max(outputs.data, 1)
            total += target.size(0)
            # Підраховуємо кількість правильних передбачень
            correct += (predicted == target).sum().item()
```

```
        # Після кожної епохи оновлюємо learning rate за допомогою
scheduler'a
        scheduler.step()

        # Обчислюємо середню втрату та точність на тренувальних даних за
епоху
        epoch_loss = running_loss / len(train_loader.dataset)
        epoch_acc  = correct / total
        train_losses.append(epoch_loss)
        train_acc.append(epoch_acc)

        # Оцінка моделі на тестових даних (без оновлення ваг)
        model.eval()  # Режим оцінки: відключається Dropout тощо
        test_loss = 0.0
        correct_test = 0
        total_test = 0

        # Вимикаємо обчислення градієнтів для економії пам'яті
        with torch.no_grad():
            for data, target in test_loader:
                outputs = model(data)
                loss = criterion(outputs, target)
                test_loss += loss.item() * data.size(0)
                _, predicted = torch.max(outputs.data, 1)
                total_test += target.size(0)
                correct_test += (predicted == target).sum().item()

        epoch_test_loss = test_loss / len(test_loader.dataset)
        epoch_test_acc  = correct_test / total_test
        test_losses.append(epoch_test_loss)
        test_acc.append(epoch_test_acc)

        # Вивід результатів поточної епохи
        print(f"Epoch {epoch+1}/{epochs}: Train loss: {epoch_loss:.4f},
Train acc: {epoch_acc:.4f} | Test loss: {epoch_test_loss:.4f}, Test acc:
{epoch_test_acc:.4f}")

    # Повертаємо словник з результатами навчання
    return {
        "train_losses": train_losses,
        "test_losses": test_losses,
        "train_acc": train_acc,
        "test_acc": test_acc
    }
```

4. **Конфігурації експериментів:**
    - **Оптимізатори:** SGD, Adam, RMSprop.
    - **Параметри learning rate:** Для SGD lr=0.01, для Adam і RMSprop lr=0.001.
    - **Scheduler:**
        - **StepLR:** Зменшує lr кожні 5 епох (gamma=0.1).
        - **CosineAnnealingLR:** Плавне зниження lr протягом 10 епох.
        - **ExponentialLR:** Експоненційне зниження lr (gamma=0.9).
    - **Регулярізація:**
        - L2 через параметр `weight_decay=1e-4`.
        - L1 через параметр `l1_lambda` (для Adam – 1e-5, для інших – 0).

5. **Виконання експериментів:**
    - Для кожної конфігурації створюється нова модель, встановлюється оптимізатор та scheduler, запускається навчання, а результати зберігаються.
6. **Групування та візуалізація:**
    - Результати групуються за типом scheduler'а.
    - Для кожного типу побудовано графіки тестової точності та тренувальних втрат, що дозволяє порівняти ефективність оптимізаторів (SGD, Adam, RMSprop) при використанні одного scheduler'а.

```
# ============================
# 4. Конфігурації експериментів
# ============================

# Тут задаємо 9 конфігурацій експериментів, комбінуючи 3 оптимізатори
(SGD, Adam, RMSprop)
# з 3 типами scheduler'ів: StepLR, CosineAnnealingLR, ExponentialLR.
# Також задаємо параметри регулярізації:
#   - weight_decay: L2 регулярізація,
#   - l1_lambda: L1 регулярізація.
experiment_configs = [
    # Конфігурації для StepLR
    {
        "name": "SGD + StepLR",
```

```
        "optimizer": "SGD",
        "lr": 0.01,
        "scheduler": "StepLR",
        "scheduler_params": {"step_size": 5, "gamma": 0.1},
        "weight_decay": 1e-4,   # L2 регулярізація
        "l1_lambda": 0.0,       # L1 регулярізація (не використовується)
    },
    {
        "name": "Adam + StepLR",
        "optimizer": "Adam",
        "lr": 0.001,
        "scheduler": "StepLR",
        "scheduler_params": {"step_size": 5, "gamma": 0.1},
        "weight_decay": 1e-4,
        "l1_lambda": 1e-5,      # Невелика L1 регулярізація
    },
    {
        "name": "RMSprop + StepLR",
        "optimizer": "RMSprop",
        "lr": 0.001,
        "scheduler": "StepLR",
        "scheduler_params": {"step_size": 5, "gamma": 0.1},
        "weight_decay": 1e-4,
        "l1_lambda": 0.0,
    },
    # Конфігурації для CosineAnnealingLR
    {
        "name": "SGD + CosineAnnealingLR",
        "optimizer": "SGD",
        "lr": 0.01,
        "scheduler": "CosineAnnealingLR",
        "scheduler_params": {"T_max": 10},
        "weight_decay": 1e-4,
        "l1_lambda": 0.0,
    },
    {
        "name": "Adam + CosineAnnealingLR",
        "optimizer": "Adam",
        "lr": 0.001,
        "scheduler": "CosineAnnealingLR",
        "scheduler_params": {"T_max": 10},
        "weight_decay": 1e-4,
        "l1_lambda": 1e-5,
    },
    {
        "name": "RMSprop + CosineAnnealingLR",
        "optimizer": "RMSprop",
        "lr": 0.001,
        "scheduler": "CosineAnnealingLR",
        "scheduler_params": {"T_max": 10},
        "weight_decay": 1e-4,
```

```
        "l1_lambda": 0.0,
    },
    # Конфігурації для ExponentialLR
    {
        "name": "SGD + ExponentialLR",
        "optimizer": "SGD",
        "lr": 0.01,
        "scheduler": "ExponentialLR",
        "scheduler_params": {"gamma": 0.9},
        "weight_decay": 1e-4,
        "l1_lambda": 0.0,
    },
    {
        "name": "Adam + ExponentialLR",
        "optimizer": "Adam",
        "lr": 0.001,
        "scheduler": "ExponentialLR",
        "scheduler_params": {"gamma": 0.9},
        "weight_decay": 1e-4,
        "l1_lambda": 1e-5,
    },
    {
        "name": "RMSprop + ExponentialLR",
        "optimizer": "RMSprop",
        "lr": 0.001,
        "scheduler": "ExponentialLR",
        "scheduler_params": {"gamma": 0.9},
        "weight_decay": 1e-4,
        "l1_lambda": 0.0,
    },
]

# ============================
# 5. Виконання експериментів
# ============================

# Створюємо словник для збереження результатів
results = {}
epochs = 10  # Кількість епох для навчання

# Проходимо по кожній конфігурації
for config in experiment_configs:
    print("\nЗапуск експерименту:", config["name"])
    # Створюємо нову модель для даної конфігурації
    model = NeuralNet()

    # Визначаємо оптимізатор відповідно до конфігурації
    if config["optimizer"] == "SGD":
        optimizer = optim.SGD(model.parameters(), lr=config["lr"],
weight_decay=config["weight_decay"])
    elif config["optimizer"] == "Adam":
```

```
        optimizer = optim.Adam(model.parameters(), lr=config["lr"],
weight_decay=config["weight_decay"])
    elif config["optimizer"] == "RMSprop":
        optimizer = optim.RMSprop(model.parameters(), lr=config["lr"],
weight_decay=config["weight_decay"])

    # Налаштовуємо scheduler (планувальник learning rate)
    if config["scheduler"] == "StepLR":
        scheduler = StepLR(optimizer, **config["scheduler_params"])
    elif config["scheduler"] == "CosineAnnealingLR":
        scheduler = CosineAnnealingLR(optimizer,
**config["scheduler_params"])
    elif config["scheduler"] == "ExponentialLR":
        scheduler = ExponentialLR(optimizer, **config["scheduler_params"])

    # Запускаємо навчання моделі з заданими параметрами
    result = train_model(model, train_loader, test_loader, optimizer,
scheduler, epochs=epochs, l1_lambda=config["l1_lambda"])
    # Зберігаємо результати у словнику
    results[config["name"]] = result

# ==============================
# 6. Групування та візуалізація результатів
# ==============================

# Групуємо результати за типом scheduler'a
grouped_results = {
    "StepLR": [],
    "CosineAnnealingLR": [],
    "ExponentialLR": []
}

# Для кожної конфігурації, якщо в назві є ім'я scheduler'a, додаємо її до
відповідної групи
for config_name, result in results.items():
    for scheduler_name in grouped_results.keys():
        if scheduler_name in config_name:
            grouped_results[scheduler_name].append((config_name, result))

# Для кожного scheduler'a будуємо графіки тестової точності та
тренувальних втрат
for scheduler_name, configs in grouped_results.items():
    # Графік тестової точності
    plt.figure(figsize=(12, 6))
    for config_name, result in configs:
        # Створюємо діапазон епох для осі X
        epochs_range = range(1, len(result["test_acc"]) + 1)
        # Отримуємо назву оптимізатора (частина до " + ")
        label = config_name.split(" + ")[0]
        plt.plot(epochs_range, result["test_acc"], label=label)
    plt.title(f"Порівняння тестової точності для {scheduler_name}")
```

```
    plt.xlabel("Епохи")
    plt.ylabel("Точність")
    plt.legend()
    plt.grid(True)
    plt.show()

    # Графік тренувальних втрат
    plt.figure(figsize=(12, 6))
    for config_name, result in configs:
        epochs_range = range(1, len(result["train_losses"]) + 1)
        label = config_name.split(" + ")[0]
        plt.plot(epochs_range, result["train_losses"], label=label)
    plt.title(f"Порівняння тренувальних втрат для {scheduler_name}")
    plt.xlabel("Епохи")
    plt.ylabel("Втрати")
    plt.legend()
    plt.grid(True)
    plt.show()
```